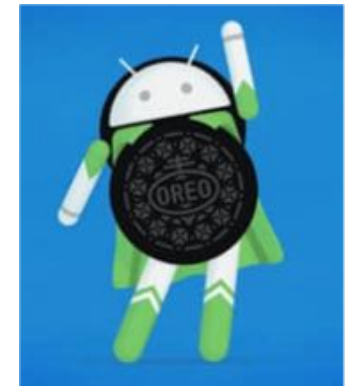




CS353 Linux Kernel



Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



Android Oreo



上海交通大学



Outline

- Android Platform Introduction
- Android Application Fundamentals
- First Android Application
- Java Review
- Kotlin Programming Language





Android Introduction



Platform Overview



上海交通大学



What is Android?



- Android is a software stack for mobile devices that includes an operating system, middleware and key applications.





OHA (Open Handset Alliance)

- A business alliance consisting of 84 companies to develop open standards for mobile devices





Features of Android

- ❑ Open Source
- ❑ Runs on all deployed cellular standards like GSM, LTE
- ❑ Provide support for multiple languages
- ❑ Attractive & flexible UI



Symbian



Android



iOS



Phones



HTC



Google



Motorola Z



小米

上海交通大学



Samsung Galaxy



华为





Tablets



Velocity Micro Cruz



Gome FlyTouch



Acer beTouch



Dawa D7



Toshiba Android SmartBook



Cisco Android Tablet



上海交通大学



MarketShare

Worldwide Smartphone Sales to End Users by Operating System in 2017 (Thousands of Units)

Operating System	2017 Units	2017 Market Share (%)	2016 Units	2016 Market Share (%)
Android	1,320,118.1	85.9	1,268,562.7	84.8
iOS	214,924.4	14.0	216,064.0	14.4
Other OS	1,493.0	0.1	11,332.2	0.8
Total	1,536,535.5	100.0	1,495,959.0	100.0

Source: Gartner (February 2018)



上海交通大学



Android History

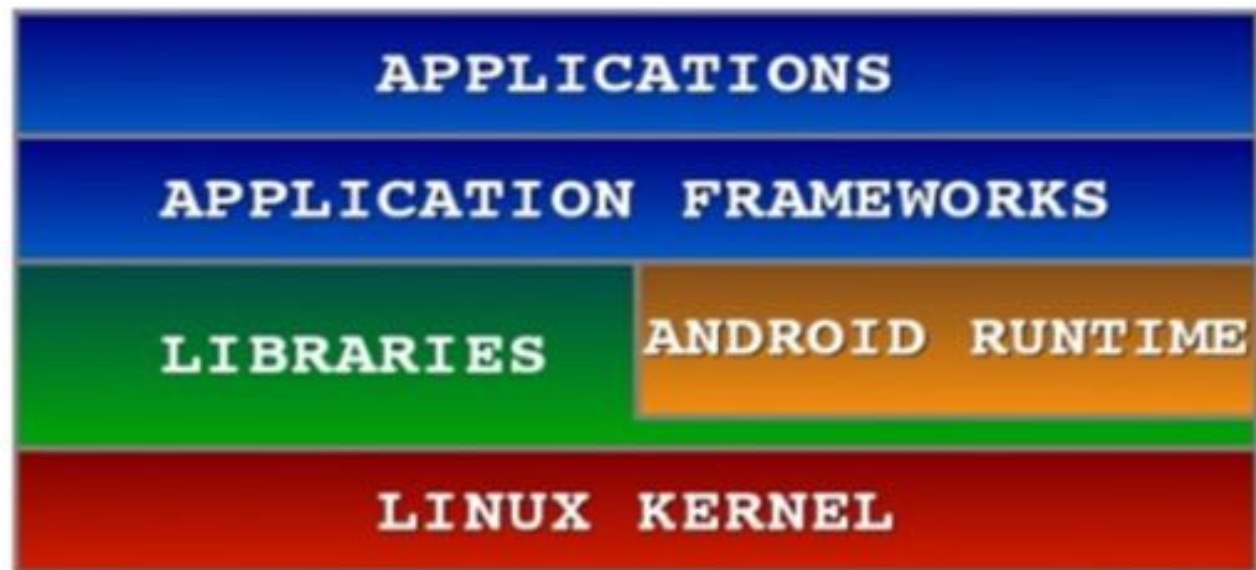


上海交通大學



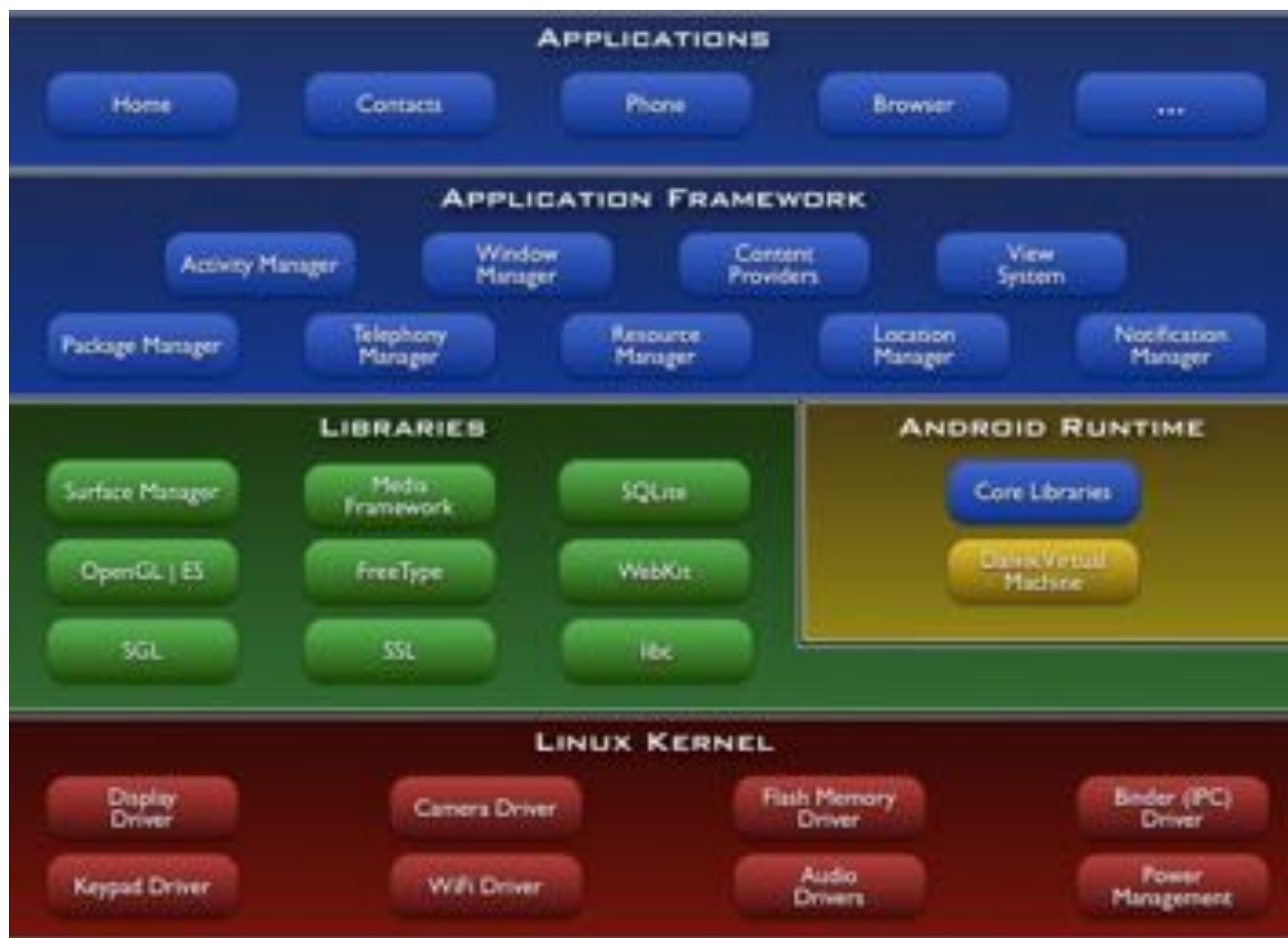
Android Architecture

- Layerd Architecture
- Each layer is a group of several programs
- Includes Operating system, Middleware, Applications
- Each layer provide services to the above layer





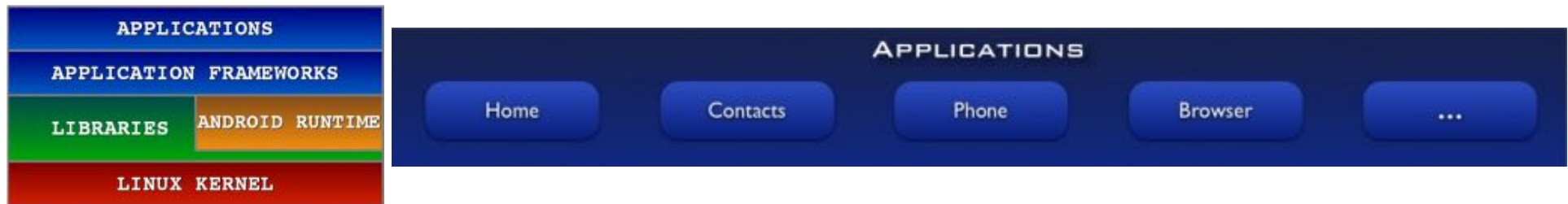
Architecture



上海交通大学



Android S/W Stack - Application

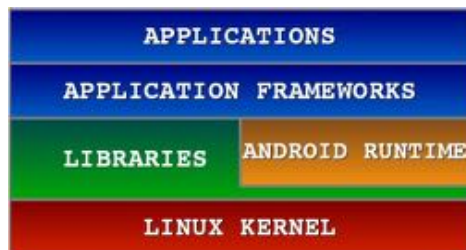


- Android provides a set of core applications:
 - ✓ Email Client
 - ✓ SMS Program
 - ✓ Calendar
 - ✓ Maps
 - ✓ Browser
 - ✓ Contacts
 - ✓ Etc

- All applications are written in Java language.



Android S/W Stack – App Framework



- ❑ Enabling and simplifying the reuse of components
 - ✓ Developers have full access to the same framework APIs used by the core applications.
 - ✓ Users are allowed to replace components.



Android S/W Stack – App Framework (Cont)



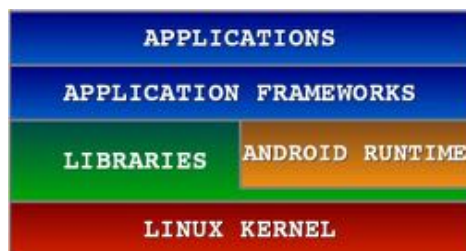
□ Features

Feature	Role
View System	Used to build an application, including lists, grids, text boxes, buttons, and embedded web browser
Content Provider	Enabling applications to access data from other applications or to share their own data
Resource Manager	Providing access to non-code resources (localized strings, graphics, and layout files)
Notification Manager	Enabling all applications to display customer alerts in the status bar
Activity Manager	Managing the lifecycle of applications and providing a common navigation backstack





Android S/W Stack - Libraries

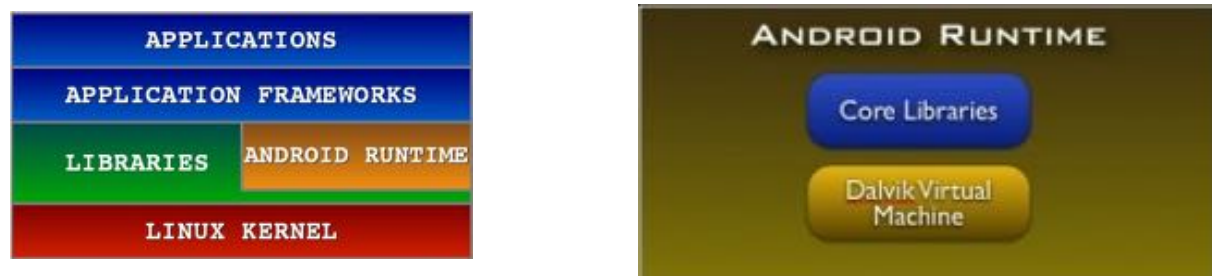


- Including a set of C/C++ libraries used by components of the Android system
- Exposed to developers through the Android application framework





Android S/W Stack - Runtime



□ Core Libraries

- ✓ Providing most of the functionality available in the core libraries of the Java language
- ✓ APIs
 - Data Structures
 - Utilities
 - File Access
 - Network Access
 - Graphics
 - Etc



上海交通大学

Android S/W Stack – Runtime (Cont)



□ Dalvik Virtual Machine

- ✓ Providing environment on which every Android application runs
 - Each Android application runs in its own process, with its own instance of the Dalvik VM.
 - Dalvik has been written such that a device can run multiple VMs efficiently.
- ✓ Register-based virtual machine

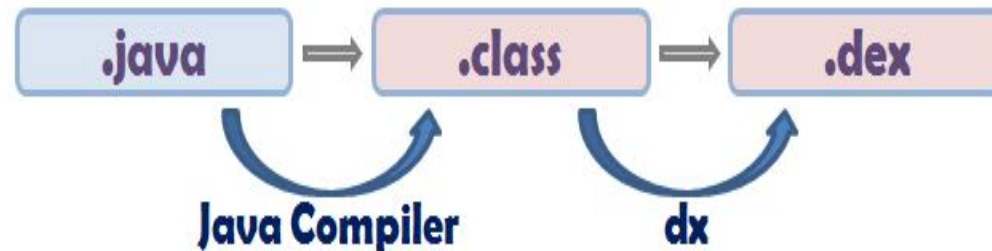


Android S/W Stack – Runtime (Cont)



□ Dalvik Virtual Machine (Cont)

- ✓ Executing the Dalvik Executable (.dex) format
 - .dex format is optimized for minimal memory footprint.
 - Compilation



- ✓ Relying on the Linux Kernel for:
 - Threading
 - Low-level memory management



Android S/W Stack – Linux Kernel



- Relying on Linux Kernel 2.6 for core system services
 - ✓ Memory and Process Management
 - ✓ Network Stack
 - ✓ Driver Model
 - ✓ Security
- Providing an abstraction layer between the H/W and the rest of the S/W stack





Outline

- Android Platform Introduction
- **Android Application Fundamentals**
- First Android Application
- Java Review
- Kotlin Programming Language





Goal

- Understand applications and their components
- Concepts:
 - activity, (活动)
 - service, (服务)
 - broadcast receiver, (广播接收者)
 - content provider, (内容提供者)
 - intent, (意向)
 - AndroidManifest (Android程序必备文件)





Applications

- Written in Java (it's possible to write native code – will not cover that here)
- Good separation (and corresponding security) from other applications:
 - Each application runs in **its own** process
 - Each process has **its own** separate VM
 - Each application is assigned an **unique Linux user ID** – by default files of that application are only visible to that application (can be explicitly exported)





Application Components

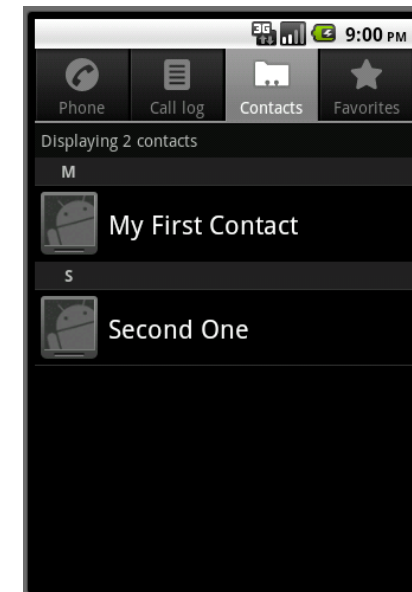
- **Activities** – visual user interface focused on a single thing a user can do
- **Services** – no visual interface – they run in the background
- **Broadcast Receivers** – receive and react to broadcast announcements
- **Content Providers** – allow data exchange between applications





Activities

- Basic component of most applications
- Most applications have several activities that start each other as needed
- Each is implemented as a subclass of the base Activity class





Activities – The View

- ❑ Each activity has a default window to draw in (although it may prompt for dialogs or notifications)
- ❑ The content of the window is a view or a group of views (derived from **View** or **ViewGroup**)
- ❑ Example of views: buttons, text fields, scroll bars, menu items, check boxes, etc.
- ❑ View(Group) made visible via **Activity.setContentView()** method.





Services

- Does not have a visual interface
- Runs in the background indefinitely
- Examples
 - Network Downloads
 - Playing Music
 - TCP/UDP Server
- You can bind to an existing service and control its operation





Broadcast Receivers

- Receive and react to broadcast announcements
- Extend the class BroadcastReceiver
- Examples of broadcasts:
 - Low battery, power connected, shutdown, timezone changed, etc.
 - Other applications can initiate broadcasts





Content Providers

- ❑ Makes some of the application data available to other applications
- ❑ It's the **only way to transfer data** between applications in Android (no shared files, shared memory, pipes, etc.)
- ❑ Extends the class `ContentProvider`;
- ❑ Other applications use a **ContentResolver** object to access the data provided via a `ContentProvider`





Intents

- An intent is an **Intent** object with a message content.
- Activities, services and broadcast receivers are started by intents. ContentProviders are started by ContentResolvers:
 - An **activity** is started by Context.startActivity(Intent intent) or Activity.startActivityForResult(Intent intent, int requestCode)
 - A **service** is started by Context.startService(Intent service)
 - An application can initiate a **broadcast** by using an Intent in any of Context.sendBroadcast(Intent intent), Context.sendOrderedBroadcast(), and Context.sendStickyBroadcast()





Shutting down components

□ Activities

- Can terminate itself via `finish()`;
- Can terminate other activities it started via `finishActivity()`;

□ Services

- Can terminate via `stopSelf()`; or `Context.stopService()`;

□ Content Providers

- Are only active when responding to `ContentResolvers`

□ Broadcast Receivers

- Are only active when responding to broadcasts





Android Manifest

- Its main purpose in life is to declare the components to the system:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest . . . >
    <application . . . >
        <activity
            android:name="com.example.project.FreneticActivity"
            android:icon="@drawable/small_pic.png"
            android:label="@string/freneticLabel"
            . . . >
        </activity>
        . . .
    </application>
</manifest>
```





Intent Filters

- Declare Intents handled by the current application (in the AndroidManifest):

```
<?xml version="1.0" encoding="utf-8"?>
<manifest . . . >
  <application . . . >
    <activity android:name="com.example.project.FreneticActivity"
      android:icon="@drawable/small_pic.png"
      android:label="@string/freneticLabel"
      . . . >
      <intent-filter . . . >
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
      <intent-filter . . . >
        <action android:name="com.example.project.BOUNCE" />
        <data android:mimeType="image/jpeg" />
        <category android:name="android.intent.category.DEFAULT" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

Shows in the
Launcher and
is the main
activity to
start

Handles JPEG
images in
some way





Outline

- Android Platform Introduction
- Android Application Fundamentals
- **First Android Application**
- Java Review
- Kotlin Programming Language





Android Introduction



Hello World

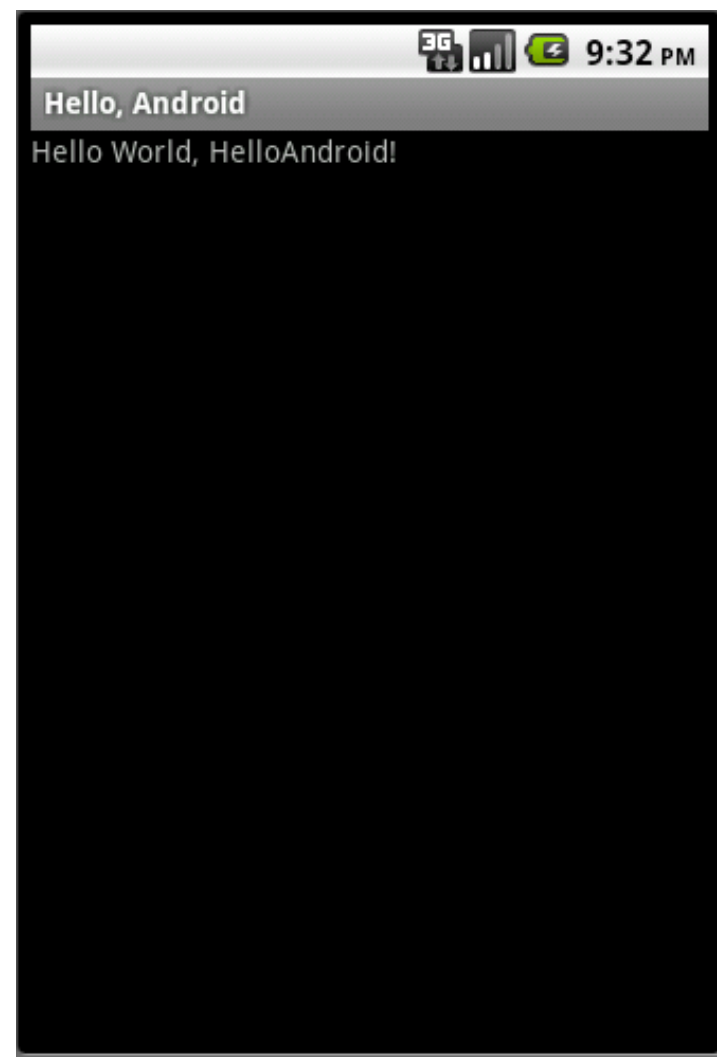


上海交通大学



Goal

- Create a very simple application
- Run it on a real device
- Run it on the emulator
- Examine its structure





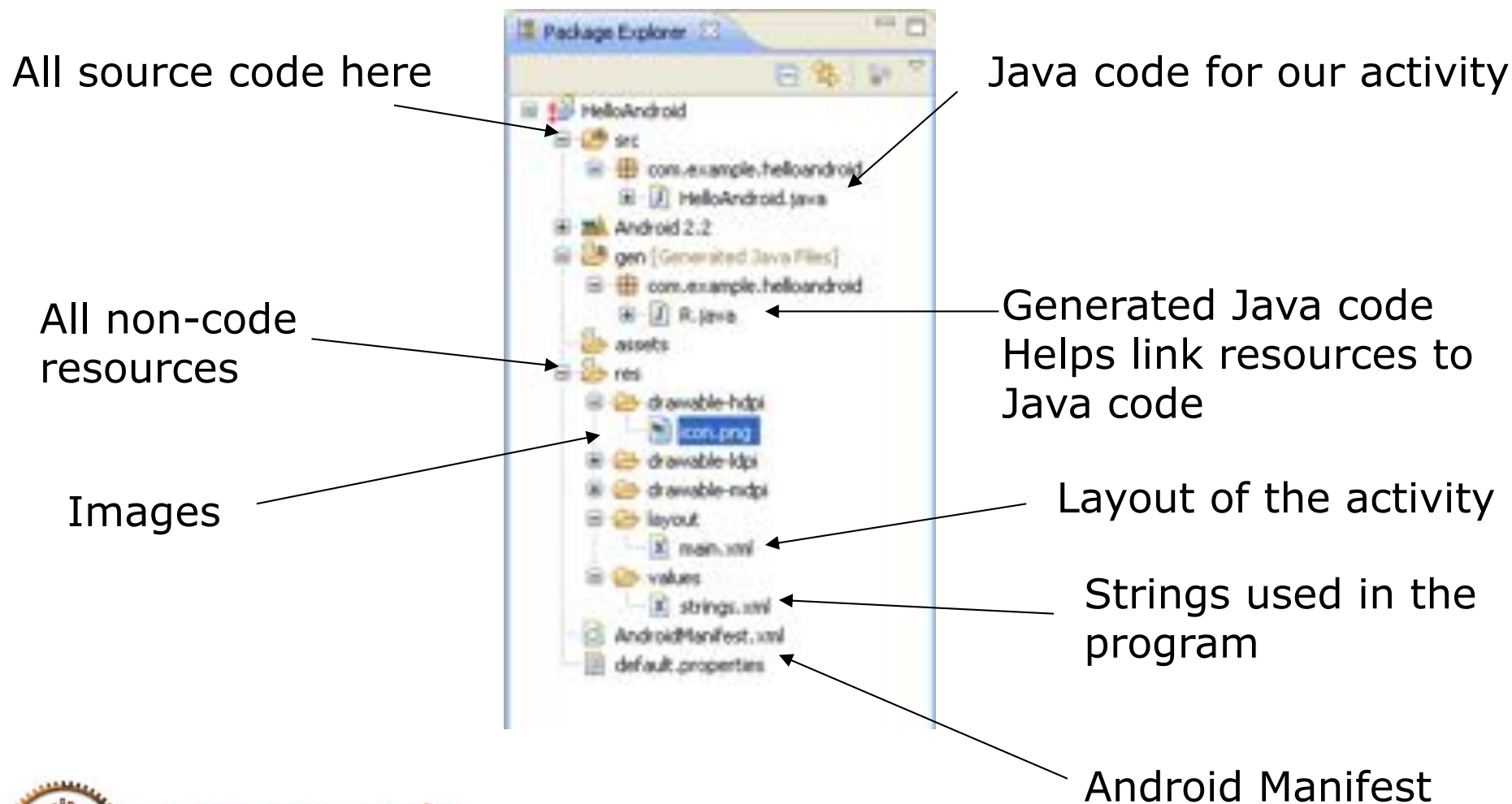
Google Tutorial

- We will follow the tutorial at:
<http://developer.android.com/resources/tutorials/hello-world.html>
- Start Eclipse (Start -> All Programs -> Eclipse)
- Create an Android Virtual Device (AVD)
- Create a New Android Project





Package Content





Android Manifest

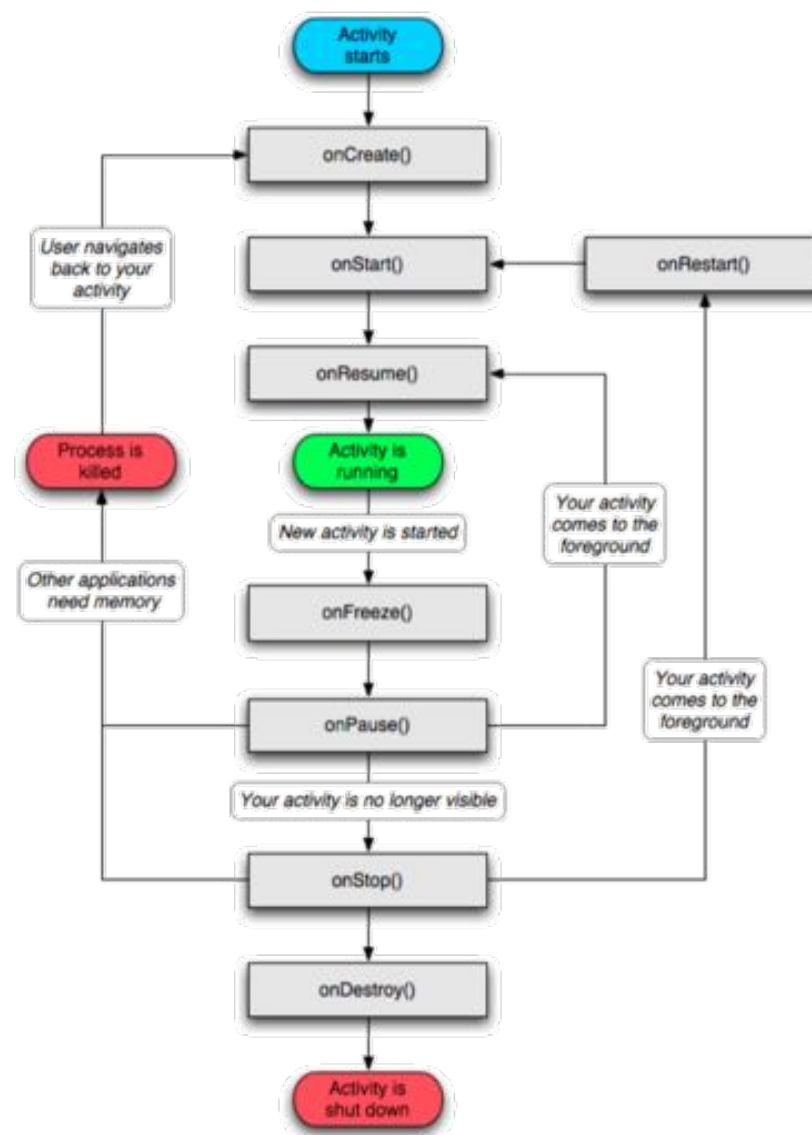
- ❑ `<?xml version="1.0" encoding="utf-8"?>`
- ❑ `<manifest xmlns:android="http://schemas.android.com/apk/res/android"`
- ❑ `package="com.example.helloandroid"`
- ❑ `android:versionCode="1"`
- ❑ `android:versionName="1.0">`
- ❑ `<application android:icon="@drawable/icon" android:label="@string/app_name">`
- ❑ `<activity android:name=".HelloAndroid"`
- ❑ `android:label="@string/app_name">`
- ❑ `<intent-filter>`
- ❑ `<action android:name="android.intent.action.MAIN" />`
- ❑ `<category android:name="android.intent.category.LAUNCHER" />`
- ❑ `</intent-filter>`
- ❑ `</activity>`
- ❑ `</application>`
- ❑ `</manifest>`





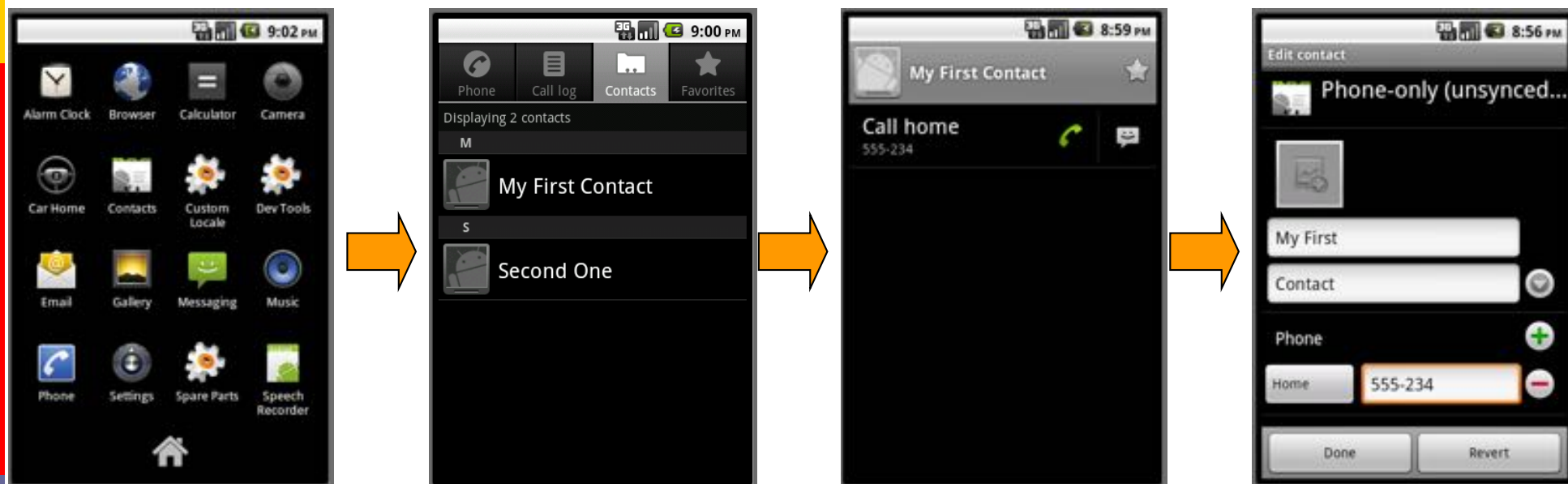
Activity

- An Android activity is focused on a single thing a user can do
- Most applications have multiple activities





Activities start each other





Revised HelloAndroid.java

```
package com.example.helloandroid;

import android.app.Activity;
import android.os.Bundle;
import android.widget.TextView;

public class HelloAndroid extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        TextView tv = new TextView(this);
        tv.setText("Hello, Android – by hand");
        setContentView(tv);
    }
}
```

Inherit
from the
Activity
Class

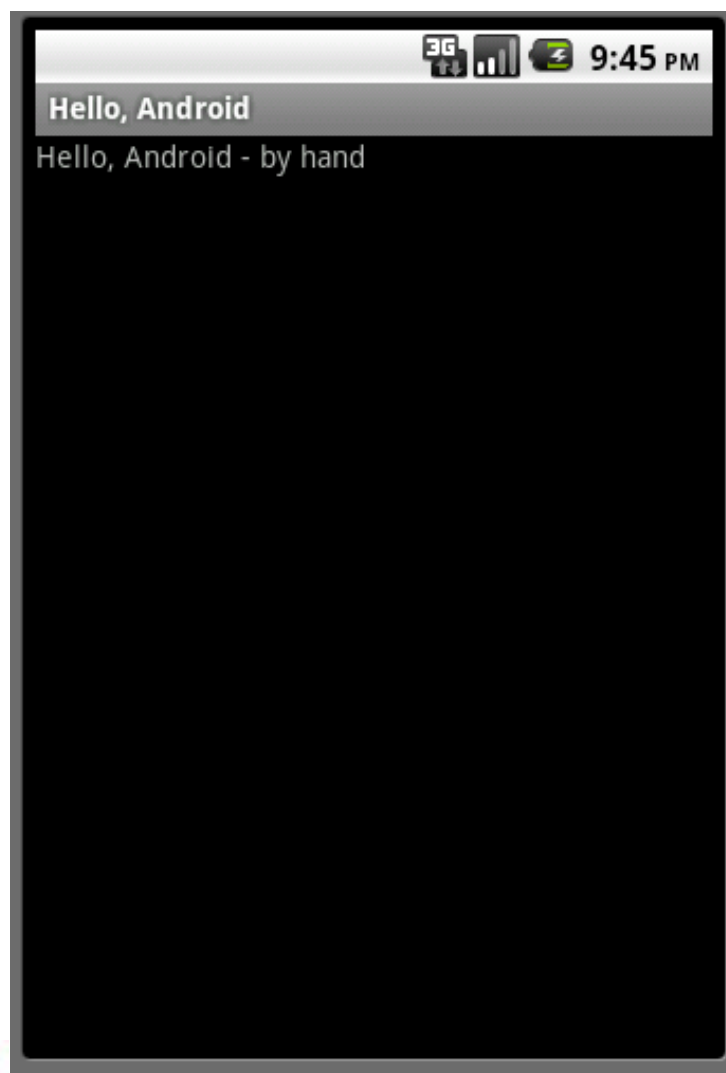
Set the view “by
hand” – from the
program



上海交通大學



Run it!



上海交通大學



/res/layout/main.xml

□ Hello world by resource

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:orientation="vertical"
  android:layout_width="fill_parent"
  android:layout_height="fill_parent"
  >
  <TextView
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:text="@string/hello"
  />
</LinearLayout>
```

Further redirection to
/res/values/strings.xml



上海交通大學



/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello">Hello World, HelloAndroid – by resources!</string>
    <string name="app_name">Hello, Android</string>
</resources>
```





HelloAndroid.java

```
package com.example.helloandroid;

import android.app.Activity;
import android.os.Bundle;
public class HelloAndroid extends Activity {

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }
}
```

Set the layout of the view as described in the main.xml layout





/gen/R.java

```
package com.example.helloandroid;

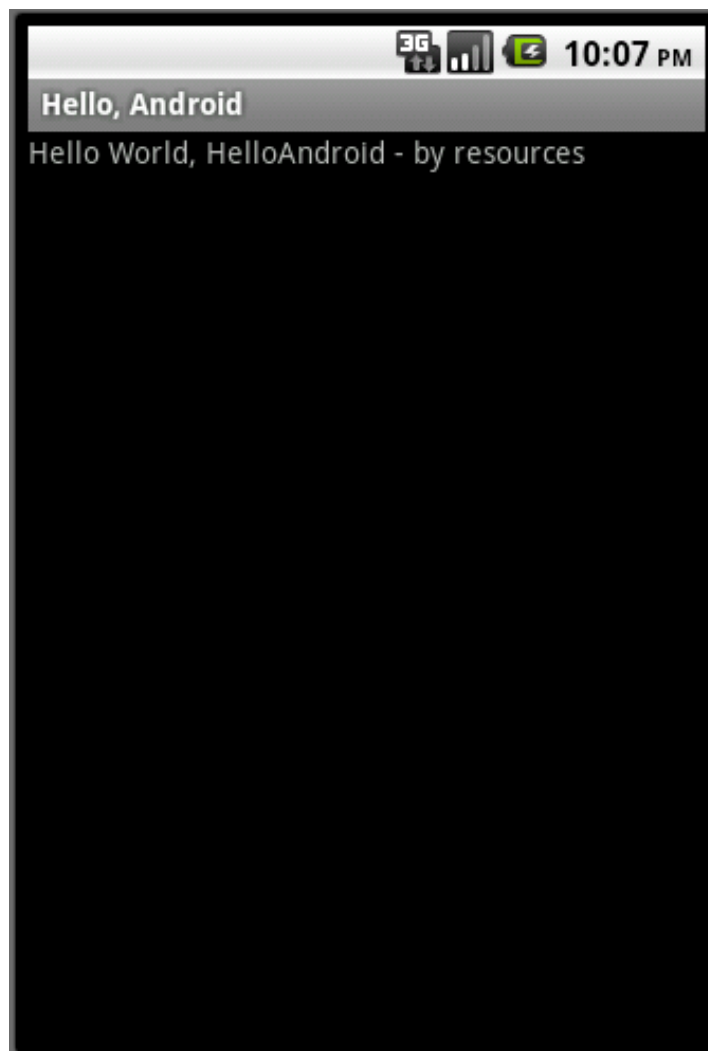
public final class R {
    public static final class attr {
    }
    public static final class drawable {
        public static final int icon=0x7f020000;
    }
    public static final class id {
        public static final int textview=0x7f050000;
    }
    public static final class layout {
        public static final int main=0x7f030000;
    }
    public static final class string {
        public static final int app_name=0x7f040001;
        public static final int hello=0x7f040000;
    }
}
```

Resource identifier





Run it!



上海交通大學



Introduce a bug

```
package com.example.helloandroid;

import android.app.Activity;
import android.os.Bundle;

public class HelloAndroid extends Activity {
    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Object o = null;
        o.toString();
        setContentView(R.layout.main);
    }
}
```





Run it!



上海交通大学



Outline

- Android Platform Introduction
- Android Application Fundamentals
- First Android Application
- **Java Review**
- Kotlin Programming Language





Java Review





Java

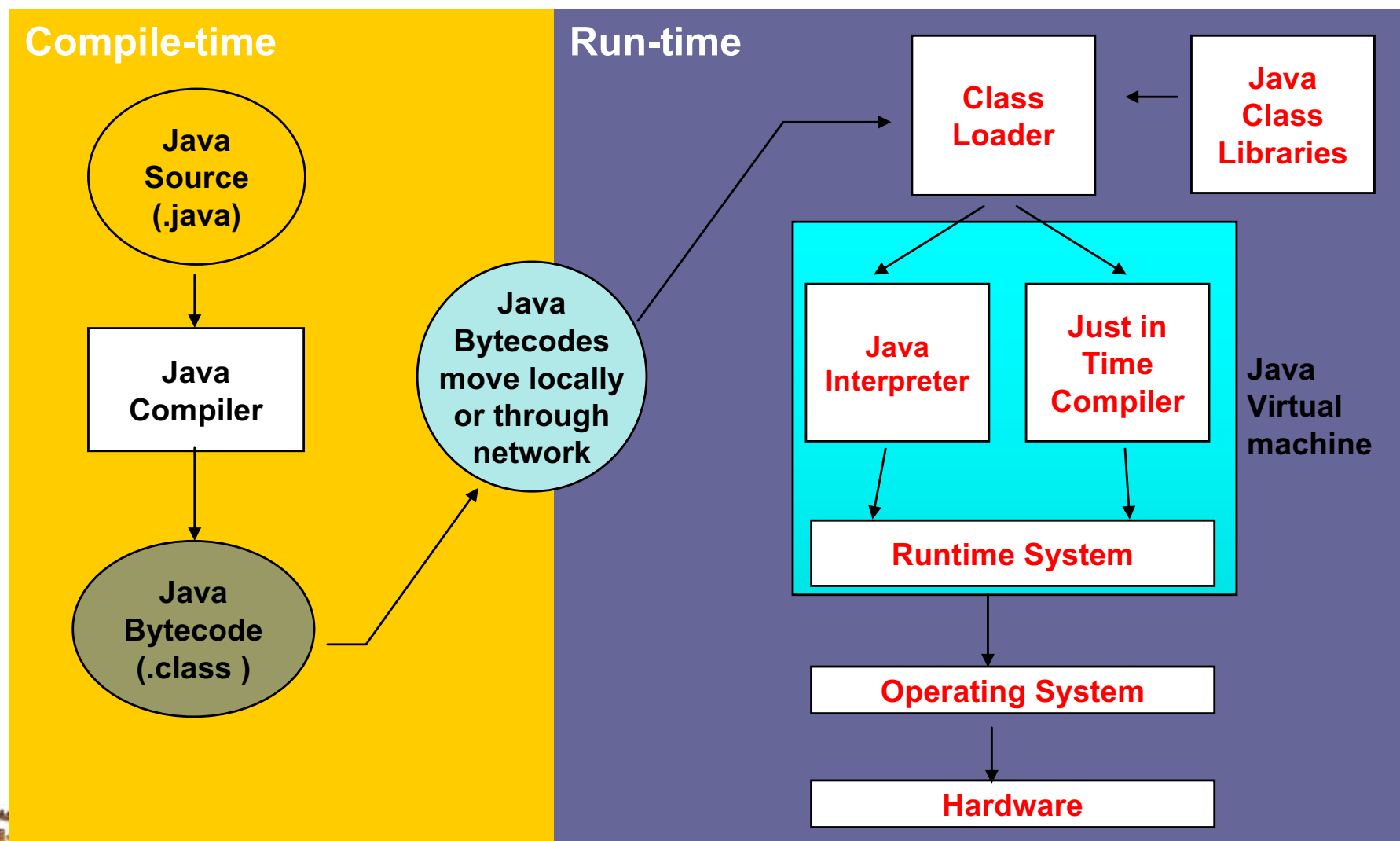
- Java is an *object-oriented* language, with a syntax similar to C
 - Structured around *objects* and *methods*
 - A method is an action or something you do with the object

- Avoid those overly complicated features of C++:
 - Operator overloading, pointer, templates, friend class, etc.





How it works...!





Getting and using java

- JDK freely download from <http://www.oracle.com>
- All text editors support java
 - Vi/vim, emacs, notepad, wordpad
 - Just save to .java file
- Eclipse IDE
 - Eclipse
 - <http://www.eclipse.org>
 - Android Development Tools (ADT) is a plugin for Eclipse





Compile and run an application

- Write java class `HolaWorld` containing a `main()` method and save in file `"HolaWorld.java"`
 - The file name *MUST* be the same as class name
- Compile with: `javac HolaWorld.java`
- Creates compiled .class file: `HolaWorld.class`
- Run the program: `java HolaWorld`
 - Notice: use the class name directly, no .class!





Outline

- Android Platform Introduction
- Android Application Fundamentals
- First Android Application
- Java Review
- Kotlin Programming Language





Features

- Suitable for Android Application
 - Compatibility
 - Performance
 - Interoperability
 - Footprint
 - Compilation Time
 - Learning Curve
 - [Kotlin Koans](#)





Scenarios

- ❑ Server side
- ❑ Andrioid
- ❑ Javascript
- ❑ Kotlin Naive





Application of Kotlin





Tools of Kotlin for Android

□ Kotlin Android Extensions

- **Anko** is a library providing a set of Kotlin-friendly wrappers around the Android APIs, and a DSL that lets you replace your layout .xml files with Kotlin code

□ Supported by Android Studio 3.0 by default





Getting started with Android & Kotlin

- Installing the Kotlin plugin
 - Bundled with Android Studio starting from Version 3.0
 - Go to *File | Settings | Plugins | Install JetBrains plugin...* and then search for and install *Kotlin*.
 - If you are looking at the "Welcome to Android Studio" screen, choose *Configure | Plugins | Install JetBrains plugin...* You'll need to restart the IDE after this completes.





Creating a project

- Choose **Start a new Android Studio project** or **File | New project**

Create New Project

 **New Project**
Android Studio

Configure your new project

Application name:

Company domain:

Package name: [Edit](#)

☐ Include C++ support

Project location:





Choose the Android Version

Create New Project

Target Android Devices

Select the form factors your app will run on

Different platforms may require separate SDKs

☒ Phone and Tablet

Minimum SDK:

Lower API levels target more devices, but have fewer features available.
By targeting API 19 and later, your app will run on approximately 73.9% of the devices that are active on the Google Play Store.
[Help me choose](#)

☐ Wear

Minimum SDK:

☐ TV

Minimum SDK:

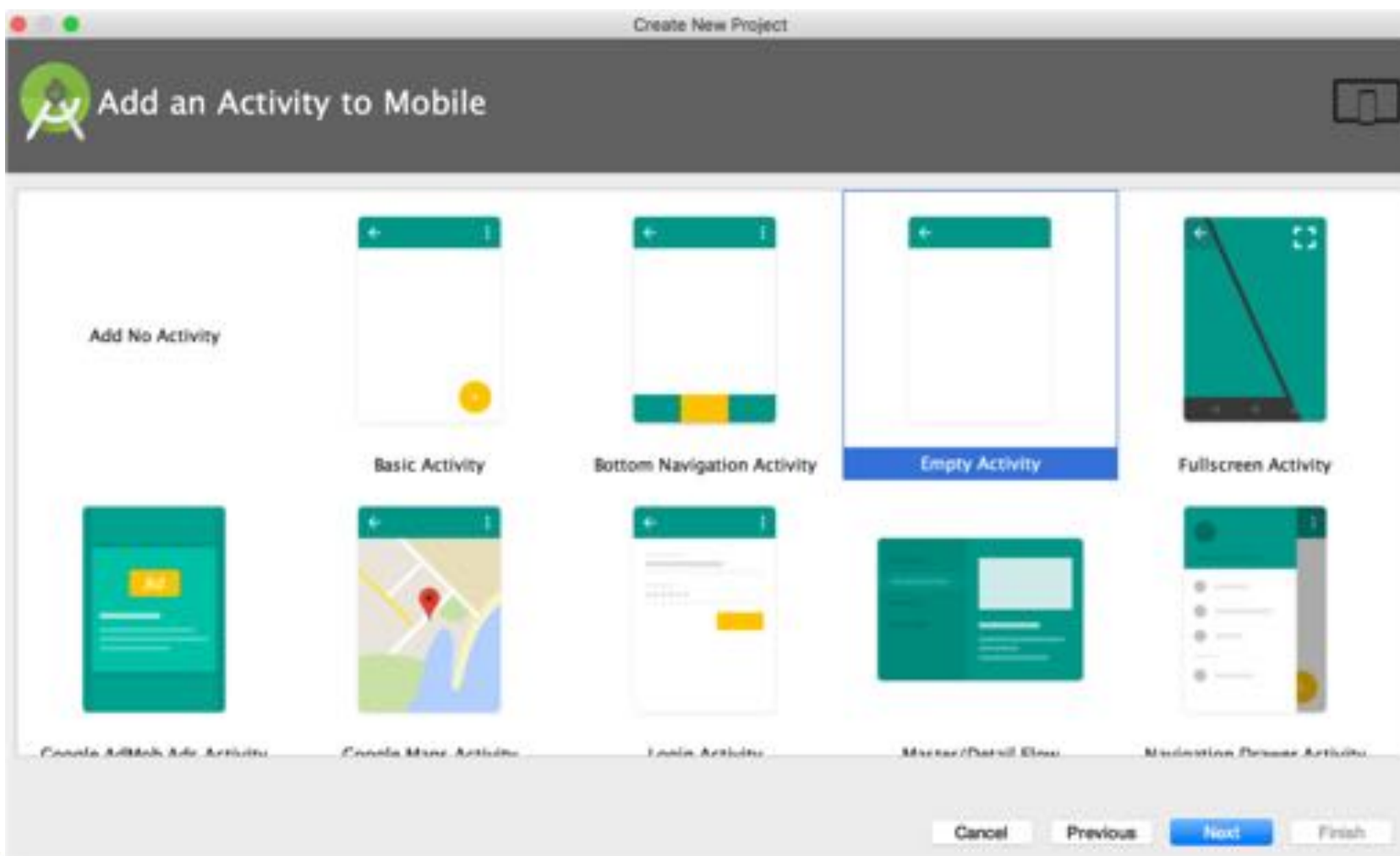
☐ Android Auto

Cancel Previous **Next** Finish





Choose creating an activity



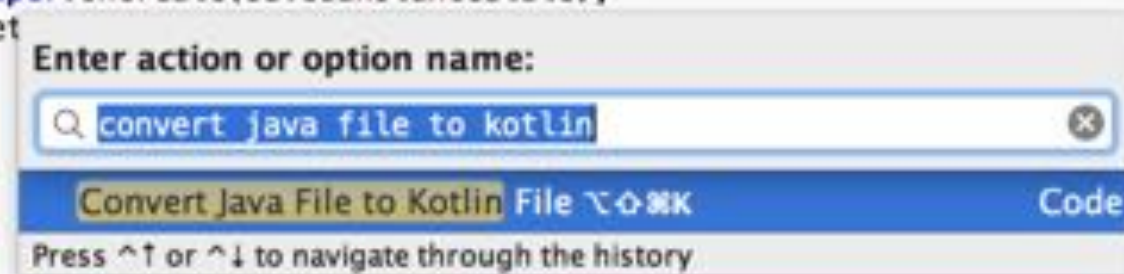


Converting Java to Kotlin

- Invoke Find Action and start typing an action name (like in a screencast below)

```
public class MainActivity extends AppCompatActivity {
```

```
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```



Before

```
class MainActivity : AppCompatActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

After



上海交通大学

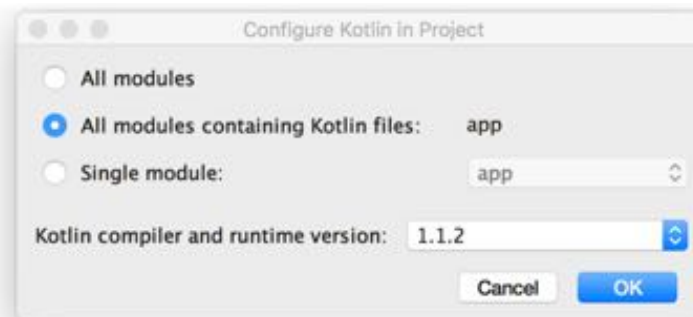


Configuring Kotlin in the project

- Android Studio shows you a prompt that Kotlin is not configured, so you can configure it



Version of Kotlin

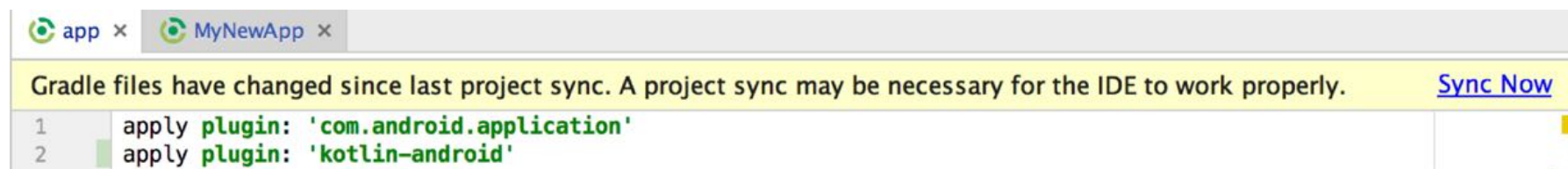


上海交通大学

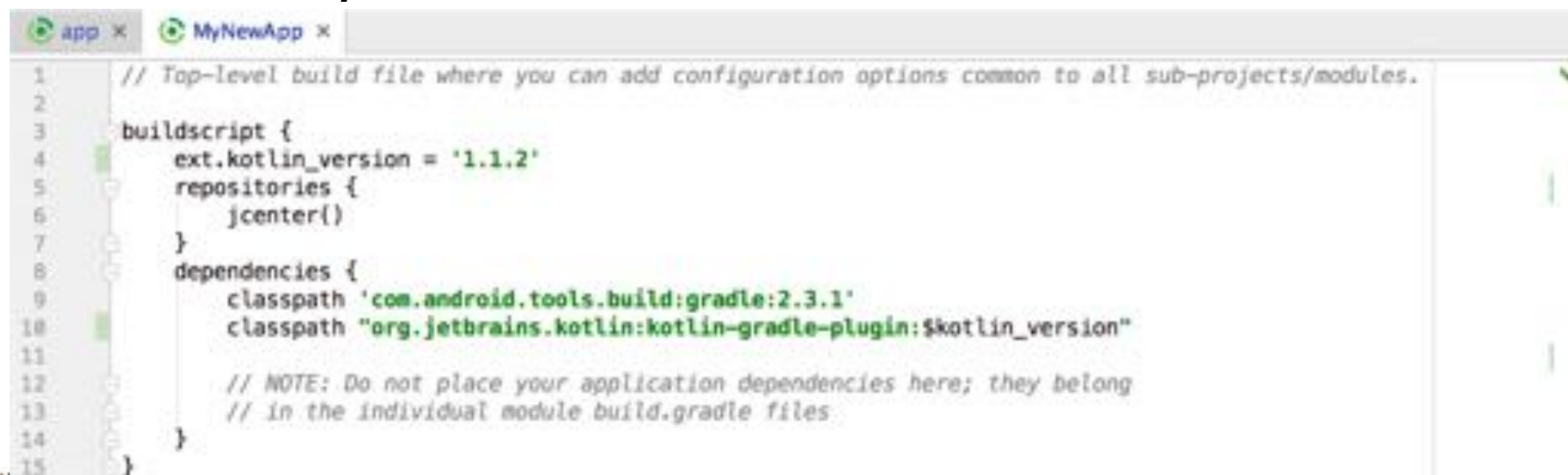


Sync project with Gradle files

- After you configure Kotlin, build.gradle file for the application should be updated



- After “Sync Now”





Building and publishing Kotlin

- Works in exactly the same way as in Java
- Kotlin has a small runtime file size: the library is approximately 943KB (as of 1.2.41).





Kotlin Basic Syntax - Packages

- Package specification should be at the top of the source file:

```
package my.demo  
  
import java.util.*  
  
// ...
```

It is not required to match directories and packages: source files can be placed arbitrarily in the file system.





Kotlin Basic Syntax - Functions

- Function having two Int parameters with Int return type:

```
//sampleStart
fun sum(a: Int, b: Int): Int {
    return a + b
}
//sampleEnd

fun main(args: Array<String>) {
    print("sum of 3 and 5 is ")
    println(sum(3, 5))
}
```

- Function with an expression body and inferred return type:

```
//sampleStart
fun sum(a: Int, b: Int) = a + b
//sampleEnd

fun main(args: Array<String>) {
    println("sum of 19 and 23 is ${sum(19, 23)}")
}
```





Basic Syntax – Variables

□ Assign-once (read-only) local variable

```
fun main(args: Array<String>) {  
    //sampleStart  
    val a: Int = 1 // 立即赋值  
    val b = 2      // 自动推断出 "Int" 类型  
    val c: Int     // 如果没有初始值类型不能省略  
    c = 3          // 明确赋值  
    //sampleEnd  
    println("a = $a, b = $b, c = $c")  
}
```

□ Mutable variable

```
fun main(args: Array<String>) {  
    //sampleStart  
    var x = 5 // 自动推断出 "Int" 类型  
    x += 1  
    //sampleEnd  
    println("x = $x")  
}
```





Basic Syntax – String templates

```
fun main(args: Array<String>) {  
    //sampleStart  
    var a = 1  
    // 模板中的简单名称：  
    val s1 = "a is $a"  
  
    a = 2  
    // 模板中的任意表达式：  
    val s2 = "${s1.replace("is", "was")}, but now is $a"  
    //sampleEnd  
    println(s2)  
}
```





Questions?



上海交通大學